

TradeNest

AI-Powered Marketplace with Secure Escrow and Smart Seller Tools

Technical Blueprint

Product vision • UI system • core stack • modular-monolith architecture • APIs • database schema • business rules • validation • security • AI

This document consolidates the project decisions made after reviewing the original SRS and SDD. Where the source documents were high-level, this blueprint adds implementation-level engineering decisions for TradeNest.

1. Executive Summary

TradeNest is a web-based multi-vendor marketplace for Buyers, Sellers, and Administrators. Its differentiators are protected escrow-style payment workflows, AI-powered recommendations and seller intelligence, fraud-risk analysis, and a dedicated seller analytics experience. The original project documents define the marketplace, AI, escrow, security, database, deployment, and role requirements at a high level; this blueprint turns those requirements into an implementation-ready design.

Core product promise

Buyers should be able to shop confidently, sellers should receive useful tools and intelligence, and financial state changes should be controlled by the backend rather than trusted to the browser.

• **Buyer:** discover products, manage cart, checkout, pay, track orders, confirm delivery, open disputes, review purchases.

• **Seller:** onboard a store, create products, manage inventory, process orders, ship, view analytics, receive AI pricing insights.

• **Admin:** manage users/products, monitor financial activity, review fraud alerts, and resolve disputes.

• **AI:** recommendations first; later pricing suggestions, behavior analysis, and fraud scoring.

• **Escrow:** successful payment is held logically until the order reaches a valid release state.

Source vs. engineering additions

The SRS/SDD explicitly support Buyer/Seller/Admin roles, product listings, recommendations, escrow payments, seller analytics, JWT, bcrypt, RBAC, an AI engine, a payment gateway, a database, cloud hosting, and testing. Exact frameworks, endpoints, detailed tables, state machines, idempotency, ledger design, and multi-layer validation were not fully specified; those are engineering decisions defined in this blueprint.

2. Locked Core Technology Stack

Layer	Decision
Frontend	Next.js 16 + React + TypeScript
Styling/UI	Tailwind CSS + shadcn/ui
Core backend	Node.js 24 LTS + NestJS + TypeScript
Architecture	Modular monolith
API	REST + OpenAPI/Swagger
Validation/contracts	Zod 4
Database	PostgreSQL 18
ORM	Prisma ORM 7
Authentication	JWT access tokens + refresh sessions + bcrypt
Authorization	RBAC + ownership/policy checks
AI service	Python + FastAPI
ML/data	scikit-learn + pandas + NumPy

Layer	Decision
Storage	S3-compatible object storage or Cloudinary
Testing	Jest + Playwright + pytest
Development	pnpm + Git/GitHub + Docker Compose

Architecture rule

Next.js owns the user experience. NestJS owns business and financial rules. PostgreSQL owns durable transactional state. Python/FastAPI owns machine-learning workloads. Zod validates data crossing TypeScript application boundaries.

3. Modular-Monolith Architecture

```
Browser / Next.js
       |
       | REST + shared Zod contracts
       v
+-----------------------------------------+
|          NestJS Modular Monolith        |
|                                         |
| Auth       Users        Catalog         |
| Cart       Orders       Payments        |
| Escrow     Shipping     Disputes        |
| Seller     Analytics    AI Bridge       |
| Admin      Notifications                |
|                                         |
|        Prisma Repository Layer          |
+------------------+----------------------+
                   |
                   v
              PostgreSQL

                   |
                   +----> FastAPI AI service
                   |
                   +----> Payment provider
                   |
                   +----> Object storage
```

Module-boundary rules

• The marketplace backend is one NestJS deployable application and one PostgreSQL database.

• A module owns its business rules and exposes an explicit service/API to other modules.

• Controllers do not call Prisma directly.

• One module should not directly mutate another module's records simply because they share a database.

• The AI service is separate because it is a specialized Python workload, not because the marketplace is being decomposed into microservices.

Suggested repository

```
tradenest/
  apps/
    web/                # Next.js
    api/                # NestJS modular monolith
  services/
    ai/                 # FastAPI / Python
  packages/
```

```
  contracts/           # Zod schemas + inferred TS types
  ui/                  # optional shared UI
  config/
infrastructure/
  docker/
docs/
pnpm-workspace.yaml
docker-compose.yml
```

4. TradeNest UI and Brand System

The visual direction is premium marketplace + fintech trust + modern SaaS. The interface should feel trustworthy and intelligent before it feels flashy. Use generous white space, restrained shadows, clear status colors, and consistent spacing.

Purpose	Hex	Usage
Primary Navy	#0B2A4A	Brand, trust, primary CTA
Primary Blue	#124C7C	Hover/active states
Teal	#0FA3A3	AI, innovation, highlights
Teal Light	#DDF7F5	AI surfaces/badges
Gold	#D6A84B	Escrow, premium trust accents
Gold Light	#FFF6DF	Escrow callouts
Background	#F7F9FC	App background
Card	#FFFFFF	Cards/panels
Border	#E2E8F0	Dividers/inputs
Primary Text	#122033	Main text
Secondary Text	#5F6B7A	Supporting text
Success	#16A34A	Completed/released
Warning	#F59E0B	Pending/review
Danger	#DC2626	Failed/disputed

Color semantics

Navy = trust. Teal = intelligence. Gold = protected transactions. Green = success. Red = dispute/error. These meanings should remain consistent across buyer, seller, and admin experiences.

Typography and component language

• Use Inter for the primary UI typeface; Manrope may be used for selected display headings.

• Buttons and inputs: approximately 10px radius; cards: 16px; modals: 20px; badges: pill radius.

• Primary buttons use navy; AI actions use teal; gold is reserved for escrow/premium trust states.

• Product cards use white surfaces, subtle borders, light shadows, strong imagery, and minimal metadata.

• Seller/admin dashboards use a sidebar + top navigation + data cards/tables/charts.

• Escrow screens use a clear timeline so users can understand where the money is.

Primary page families

```
Public / Buyer:
Home, Explore, Categories, Search, Product Detail,
Cart, Checkout, Orders, Wishlist, Profile, Payments

Seller:
```

```
Dashboard, Products, Inventory, Orders, Analytics,
AI Insights, Payments, Escrow, Disputes, Settings

Admin:
Overview, Users, Sellers, Products, Orders, Payments,
Escrow, Disputes, Fraud Alerts, Reports, Settings
```

5. API Design Conventions

• Base path: **/api/v1**.

• JSON request/response bodies.

• Bearer JWT access token for authenticated calls.

• OpenAPI/Swagger documentation exposed by NestJS.

• Financial write operations should support an **Idempotency-Key**.

• The frontend requests actions; it does not declare authoritative financial or order state.

Standard success response

```
{
  "data": {
    "id": "..."
  }
}
```

Collection response

```
{
  "data": [],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 130,
    "totalPages": 7
  }
}
```

Standard error

```
{
  "error": {
    "code": "PRODUCT_OUT_OF_STOCK",
    "message": "The requested quantity is not available.",
    "details": { "productId": "..." },
    "requestId": "req_..."
  }
}
```

5.1 Authentication

Method	Endpoint	Access / Purpose
POST	/auth/register	Public - create Buyer/Seller-capable account
POST	/auth/login	Public - authenticate
POST	/auth/refresh	Refresh session - rotate access
POST	/auth/logout	Authenticated - revoke session
POST	/auth/logout-all	Authenticated - revoke all sessions
GET	/auth/me	Authenticated - current identity
POST	/auth/change-password	Authenticated

ADMIN must never be publicly registerable.

5.2 Users and Addresses

Method	Endpoint	Access / Purpose
GET	/users/me	Read profile
PATCH	/users/me	Update profile
DELETE	/users/me	Account deletion workflow
GET	/users/me/addresses	List addresses
POST	/users/me/addresses	Create address
PATCH	/users/me/addresses/:addressId	Update owned address
DELETE	/users/me/addresses/:addressId	Delete owned address

5.3 Seller

Method	Endpoint	Access / Purpose
POST	/seller/onboard	Create/activate seller profile
GET	/seller/profile	Seller profile
PATCH	/seller/profile	Update seller profile
GET	/seller/dashboard	Seller summary
GET	/seller/orders	Seller order list
GET	/seller/products	Seller product list
GET	/seller/analytics	Seller analytics

5.4 Categories and Products

Method	Endpoint	Access / Purpose
GET	/categories	Public categories
POST	/categories	Admin
PATCH	/categories/:id	Admin
GET	/products	Public search/filter
GET	/products/:slug	Public product detail
POST	/products	Seller
PATCH	/products/:id	Owning seller
DELETE	/products/:id	Owning seller/Admin
POST	/products/:id/images	Owning seller
PATCH	/products/:id/inventory	Owning seller
POST	/products/:id/publish	Owning seller

Method	Endpoint	Access / Purpose
POST	/products/:id/archive	Owning seller

5.5 Cart and Checkout

Method	Endpoint	Access / Purpose
GET	/cart	Buyer
POST	/cart/items	Buyer - add item
PATCH	/cart/items/:itemId	Buyer - quantity
DELETE	/cart/items/:itemId	Buyer
DELETE	/cart	Buyer - clear cart
POST	/checkout/quote	Buyer - authoritative quote
POST	/checkout	Buyer - create order/payment intent

5.6 Orders, Shipping, Payments and Escrow

Method	Endpoint	Access / Purpose
GET	/orders	Buyer orders
GET	/orders/:id	Authorized participant/Admin
POST	/orders/:id/cancel	Buyer - if eligible
POST	/orders/:id/confirm-delivery	Buyer
POST	/orders/:id/shipment	Seller
GET	/orders/:id/shipment	Buyer/Seller
POST	/orders/:orderId/payments	Buyer
GET	/payments/:id	Owner/Admin
POST	/payments/webhooks/:provider	Payment provider
GET	/orders/:id/escrow	Buyer/Seller/Admin
POST	/escrows/:id/release	Internal/Admin
POST	/escrows/:id/refund	Internal/Admin

5.7 Disputes, Reviews, Wishlist

Method	Endpoint	Access / Purpose
POST	/orders/:id/disputes	Buyer
GET	/disputes/:id	Participant/Admin
POST	/disputes/:id/messages	Participant/Admin
POST	/disputes/:id/evidence	Buyer/Seller

Method	Endpoint	Access / Purpose
POST	/admin/disputes/:id/resolve	Admin
GET	/products/:id/reviews	Public
POST	/orders/:orderId/reviews	Verified buyer
GET	/wishlist	Buyer
POST	/wishlist/:productId	Buyer
DELETE	/wishlist/:productId	Buyer

5.8 AI, Analytics and Admin

Method	Endpoint	Access / Purpose
POST	/events	Behavior event ingestion
GET	/ai/recommendations	Personalized recommendations
GET	/ai/recommendations/products/:id	Similar products
POST	/seller/products/:id/price-suggestion	Seller AI insight
GET	/seller/ai-insights	Seller AI insight feed
GET	/seller/analytics/overview	Seller KPI overview
GET	/seller/analytics/revenue	Revenue analytics
GET	/admin/dashboard	Admin summary
GET	/admin/users	Admin
GET	/admin/payments	Admin
GET	/admin/escrows	Admin
GET	/admin/fraud-alerts	Admin

6. Database Schema

TradeNest uses one PostgreSQL database. Prisma is the application persistence layer, while PostgreSQL constraints remain the final integrity boundary.

Table	Core fields / purpose
users	id, email, password_hash, name, role, status, timestamps
user_sessions	refresh token hash, expiry, revocation, IP/user agent
seller_profiles	user_id, store_name, slug, status, rating
addresses	user-owned shipping addresses
categories	nested category tree via parent_id
products	seller/category, name, slug, description, price, currency, SKU, status
product_images	object-storage URL/key, order, primary flag
inventory	available_quantity, reserved_quantity
carts	one active cart per user
cart_items	cart/product/quantity
orders	buyer, totals, status, shipping-address snapshot
order_items	product/seller snapshots, quantity, unit/total price
payments	provider IDs, amount, status, failure data
escrows	order/payment, amount, current escrow state
escrow_transactions	HOLD/RELEASE/REFUND ledger entries
shipments	carrier, tracking, shipping status/timestamps
disputes	reason, status, resolution, participants
dispute_messages	conversation history
dispute_evidence	evidence file references
wishlist_items	user/product relation
reviews	verified-purchase review tied to order item
user_events	behavior signals for recommendations
ai_recommendations	optional recommendation cache/model version
fraud_assessments	risk score, level, reasons, model version
notifications	user notifications
audit_logs	sensitive action history

6.1 Key table details

```
users
  id UUID PK
  email VARCHAR UNIQUE NOT NULL
```

```
  password_hash VARCHAR NOT NULL
  name VARCHAR NOT NULL
  role USER_ROLE NOT NULL
  status USER_STATUS NOT NULL
  email_verified_at TIMESTAMP NULL
  created_at / updated_at TIMESTAMP

products
  id UUID PK
  seller_id UUID FK
  category_id UUID FK
  name VARCHAR NOT NULL
  slug VARCHAR UNIQUE
  description TEXT
  price NUMERIC(12,2) CHECK (price >= 0)
  currency CHAR(3)
  sku VARCHAR
  status PRODUCT_STATUS
  created_at / updated_at

inventory
  product_id UUID PK/FK
  available_quantity INTEGER CHECK (available_quantity >= 0)
  reserved_quantity INTEGER CHECK (reserved_quantity >= 0)

orders
  id UUID PK
  order_number UNIQUE
  buyer_id UUID FK
  status ORDER_STATUS
  currency CHAR(3)
  subtotal / shipping_amount / discount_amount / total_amount NUMERIC
  shipping_address JSONB
  paid_at / completed_at / cancelled_at TIMESTAMP NULL

order_items
  id UUID PK
  order_id UUID FK
  product_id UUID FK
  seller_id UUID FK
  product_name / sku snapshot
  quantity INTEGER CHECK (quantity > 0)
  unit_price / total_price NUMERIC
```

```
payments
  id UUID PK
  order_id UUID FK
  buyer_id UUID FK
  provider
  provider_payment_id UNIQUE
  status PAYMENT_STATUS
  amount NUMERIC CHECK (amount > 0)
  currency CHAR(3)
  failure_code / failure_message
  paid_at TIMESTAMP NULL

escrows
  id UUID PK
  order_id UUID UNIQUE FK
  payment_id UUID FK
  status ESCROW_STATUS
  amount NUMERIC
  held_at / released_at / refunded_at TIMESTAMP NULL

escrow_transactions
  id UUID PK
  escrow_id UUID FK
  type HOLD | RELEASE | REFUND | ADJUSTMENT
  amount NUMERIC
  reference
  created_at
```

```
disputes
  id UUID PK
  order_id UUID UNIQUE FK
  buyer_id / seller_id UUID FK
  reason / description
  status DISPUTE_STATUS
  resolution DISPUTE_RESOLUTION NULL
  resolved_by UUID NULL
  opened_at / resolved_at
```

6.2 Important constraints and indexes

• UNIQUE(users.email), UNIQUE(products.slug), UNIQUE(payments.provider_payment_id).

• UNIQUE(cart_id, product_id) prevents duplicate cart rows.

• UNIQUE(order_item_id) on reviews prevents repeated review of the same purchased item.

• CHECK price >= 0, quantity > 0, rating between 1 and 5, inventory quantities >= 0.

• Indexes on product seller/category/status, order buyer/status/created_at, payment provider ID, escrow order/status, dispute status, and user-event dimensions.

• Historical order items store product name, SKU and price snapshots so later catalog changes do not rewrite history.

7. Domain State Machines

Order state

```
PENDING_PAYMENT
      |
      | payment succeeds
      v
     PAID
      |
      v
  PROCESSING
      |
      v
   SHIPPED
      |
      v
  DELIVERED
      |
      v
  COMPLETED

Alternative terminal/branch states:
CANCELLED, DISPUTED, REFUNDED
```

Escrow state

```
PENDING
   |
   | verified payment
   v
  HELD
   |
   | seller ships
   v
SELLER_SHIPPED
   |
   | buyer confirms
   v
BUYER_CONFIRMED
   |
   v
RELEASED

Dispute branch:
HELD / SELLER_SHIPPED -> DISPUTED -> UNDER_REVIEW
                                  -> RELEASED
                                  -> REFUNDED
```

Product state

```
DRAFT -> ACTIVE -> ARCHIVED
          |
          +----> SUSPENDED (admin moderation)
```

Invalid transitions

Clients must not be allowed to PATCH arbitrary status values. Use command-style endpoints such as cancel, create shipment, confirm delivery, resolve dispute, release, and refund. Invalid transitions return a conflict such as 409 INVALID_STATE_TRANSITION.

8. Core Business Logic

8.1 Registration and login

```
Registration:
Zod validate -> normalize email -> check uniqueness
-> bcrypt hash -> create user -> create session -> issue tokens

Login:
validate -> find active user -> bcrypt compare
-> check account status -> create session -> issue tokens
```

Refresh tokens are stored as hashes, rotated on refresh, and revocable per session or globally. A suspended/banned user must not authenticate even with a correct password.

8.2 Product creation and ownership

• Only an ACTIVE seller can create products.

• Category must exist; price and stock must satisfy validation.

• Create Product + Inventory in one transaction.

• New products begin as DRAFT and become ACTIVE only through publish rules.

• Seller role is insufficient for editing: product.sellerId must equal the authenticated user unless an authorized admin action is being performed.

8.3 Cart and authoritative pricing

A cart is convenient state, not a price or inventory guarantee. At checkout the backend reloads current products, seller status, prices, and stock. The browser never submits a trusted final price.

8.4 Checkout transaction

```
BEGIN TRANSACTION

load cart
lock / atomically update inventory

for each item:
  verify product ACTIVE
  verify seller ACTIVE
  verify sufficient inventory
  read current authoritative price

calculate subtotal / shipping / discount / total
reserve inventory
create order
create immutable order-item snapshots
create payment intent/record

COMMIT
```

If any step fails, the entire checkout transaction rolls back. Inventory reservation must be concurrency-safe; an atomic update or row lock prevents two buyers from purchasing the same final stock.

8.5 Payment success

```
Payment provider webhook
  -> verify signature
  -> find provider transaction
  -> idempotency check
  -> validate expected amount/currency

BEGIN TRANSACTION
  Payment = SUCCEEDED
  Order = PAID
  Escrow = HELD
  append escrow HOLD ledger entry
COMMIT
```

The client can never declare a payment successful. Only verified provider state and backend logic can do that.

8.6 Shipping and delivery confirmation

• Seller may create shipment only for an eligible paid/processing order with escrow held.

• Shipping transitions Order to SHIPPED and Escrow to SELLER_SHIPPED.

• Buyer confirmation verifies buyer ownership, eligible order state, unreleased escrow, and absence of an active dispute.

• Valid confirmation completes the order and releases escrow through a transaction and ledger entry.

8.7 Disputes

```
Eligible paid order
   |
Buyer opens dispute
   v
Dispute = OPEN
Escrow = DISPUTED
   |
Admin review + evidence
   |
   +--> REFUND_BUYER
   |       Escrow = REFUNDED
   |       Order = REFUNDED
   |
   +--> RELEASE_TO_SELLER
           Escrow = RELEASED
           Order = COMPLETED
```

All refund/release decisions must be audited. While a dispute is open, automatic release is blocked.

8.8 Cancellation and inventory restoration

Before successful payment, an eligible buyer cancellation can set the order to CANCELLED and release reserved inventory. After funds are captured/held, cancellation becomes a refund workflow rather than a simple status change.

8.9 Reviews

Only a buyer with a completed order item may review that product. Rating must be 1-5 and the database enforces one review per purchased order item.

9. AI and Analytics Design

AI is deliberately staged. The transactional marketplace should work before advanced ML is introduced.

9.1 Recommendation MVP

```
Behavior signals (example weights)
PRODUCT_VIEW     +1
WISHLIST         +3
ADD_TO_CART      +4
PURCHASE         +8

events -> preference profile -> candidate products
       -> similarity/ranking -> recommendations
```

The browser sends behavior events to NestJS. NestJS owns the public API and may call FastAPI internally. FastAPI should not be exposed directly to the browser.

9.2 Fraud-risk MVP

Start with rules + risk scoring rather than claiming a sophisticated trained fraud model without data. Signals may include repeated payment failures, unusually high first orders, abnormal transaction frequency, and suspicious account/device patterns. High risk should normally trigger review, not automatic accusation.

9.3 Smart pricing

Pricing suggestions may use category, demand, views, conversion, sales history, and comparable products. The AI may recommend a range, but it must never silently change a seller's price.

9.4 Seller analytics

• Revenue, order count, products sold, average order value, conversion rate.

• Top products, low-stock products, order-status distribution.

• Escrow balance/status visibility.

• AI insights displayed with a distinct teal visual language.

10. Validation Strategy: Frontend, Backend, Database

Validation exists at all three layers, but for different reasons.

Layer	Responsibility	Examples
Frontend	Fast UX feedback	required fields, email format, password confirmation
Backend Zod	Reject untrusted malformed input	types, lengths, enums, numeric ranges
Backend services	Business validation	ownership, seller active, stock availability, state transitions
Database	Hard integrity guarantee	NOT NULL, UNIQUE, FK, CHECK, transactions

10.1 Shared Zod contracts

```
packages/contracts/
  auth/
    register.ts
    login.ts
```

```
  products/
    create-product.ts
    update-product.ts
    product-query.ts
  checkout/
  orders/
  payments/
  disputes/
  seller/
  common/
```

```
export const CreateProductSchema = z.object({
  name: z.string().trim().min(3).max(200),
  description: z.string().trim().min(20).max(5000),
  categoryId: z.string().uuid(),
  price: z.number().positive().max(99_999_999),
  stock: z.number().int().min(0).max(1_000_000),
  sku: z.string().trim().min(1).max(100)
});

export type CreateProductInput =
  z.infer<typeof CreateProductSchema>;
```

The same core schema can support frontend forms and backend request validation. Backend validation is still mandatory because clients can bypass the UI.

10.2 Business validation examples

• Zod can validate that quantity is a positive integer; the service/database must decide whether enough inventory exists.

• Zod can validate a UUID; the service must decide whether that category exists.

• RBAC can validate SELLER; policy logic must verify the seller owns the product.

• A status enum may be valid syntactically, but the state machine decides whether a transition is legal.

10.3 Database constraints

```
CHECK (price >= 0)
CHECK (quantity > 0)
CHECK (rating BETWEEN 1 AND 5)

UNIQUE (email)
UNIQUE (cart_id, product_id)
UNIQUE (provider_payment_id)

FOREIGN KEY (seller_id) REFERENCES users(id)
```

Zod gives friendly application errors; PostgreSQL guarantees integrity under concurrency. Neither replaces the other.

11. Security and Financial Integrity

• bcrypt password hashing; never store plaintext passwords.

• Short-lived JWT access tokens plus longer refresh sessions; store only refresh-token hashes.

• RBAC plus resource ownership/policy checks.

• Verify payment webhook signatures and expected amount/currency.

• Never render untrusted raw HTML from product descriptions, reviews, seller bios, or dispute messages.

• Never return password hashes or internal secrets in API responses.

• Use database transactions for checkout, payment success, refunds, escrow release, dispute resolution, and inventory changes.

• Create audit logs for refunds, escrow releases, dispute resolutions, suspensions, and other sensitive admin actions.

• Use idempotency for checkout/payment/refund/release operations.

Money handling

Avoid ordinary JavaScript floating-point arithmetic for authoritative financial calculations. PostgreSQL NUMERIC is acceptable for the project; alternatively use integer minor units consistently. All totals are computed on the backend from authoritative prices.

Idempotency example

```
Client sends:
Idempotency-Key: a5cc24ea-...

Backend stores:
key + user + operation + original response

Retry with same key:
return original result
do not create a second order/payment/refund
```

12. Multi-Seller Checkout Decision

A true marketplace cart may contain products from multiple sellers. This complicates shipments, seller settlement, disputes, and escrow because each seller may complete independently.

Recommended MVP

Initially restrict checkout to one seller per order while the payment/escrow lifecycle is stabilized. Then evolve to a parent-order model with seller sub-orders.

```
Future:
Parent Order
   |
   +-- Seller Order A -> shipment A -> escrow/settlement A
   |
   +-- Seller Order B -> shipment B -> escrow/settlement B
```

13. Recommended Development Sequence

Phase	Scope
1. Foundation	Monorepo, NestJS, Next.js, PostgreSQL, Prisma, Zod, Auth, Users, RBAC

Phase	Scope
2. Catalog	Seller onboarding, categories, products, images, inventory
3. Shopping	Cart, addresses, checkout quote, orders
4. Financial	Payment abstraction, sandbox provider, escrow, ledger
5. Fulfillment	Shipping, delivery confirmation, cancellation, refunds, disputes
6. Intelligence	User events, recommendations, seller analytics
7. Advanced AI	Pricing suggestions, fraud scoring, improved recommendation models
8. Hardening	E2E tests, audit, rate limits, observability, deployment

Do not begin with advanced AI. First make the complete transaction path reliable: register -> seller/product -> cart -> checkout -> payment -> escrow -> shipment -> confirmation/release -> dispute/refund.

14. Testing Strategy

• **Jest:** NestJS unit and integration tests for services, policies, state transitions, and repositories.

• **Playwright:** end-to-end buyer/seller/admin workflows through the Next.js UI.

• **pytest:** FastAPI and ML pipeline tests.

• **Database integration tests:** uniqueness, foreign keys, inventory concurrency, transactions, rollback behavior.

• **Payment tests:** duplicate webhooks, invalid signatures, mismatched amounts, failed payments, refunds.

• **Escrow tests:** illegal releases, active disputes, duplicate releases, refund/release exclusivity.

15. Development and Deployment Shape

```
Local development

Browser
   |
Next.js :3000
   |
NestJS API :4000
   |---------------- PostgreSQL :5432
   |
   +---------------- FastAPI :8000
   |
   +---------------- Payment sandbox
   |
   +---------------- Object storage
```

Docker Compose can initially run PostgreSQL and supporting services. The application can later be containerized for deployment. The SDD's future scalable deployment direction (cloud hosting, separate database server, load balancing) remains compatible with this modular-monolith design.

16. Non-Negotiable Engineering Principles

1. The frontend requests actions; it never declares authoritative truth.

2. Payment success comes from verified provider state, not the browser.

3. Prices and totals are calculated by the backend.

4. Order, payment, and escrow are separate state machines.

5. Financial transitions are transactional, idempotent, and audited.

6. Zod validates boundaries; services validate business rules; PostgreSQL enforces integrity.

7. Role checks are not enough: resource ownership and domain policy checks are required.

8. Modules communicate through explicit services/interfaces, not random cross-table updates.

9. Prisma models are not public API contracts.

10. AI recommendations assist users; they do not silently mutate critical business data.

11. Do not prematurely split the core marketplace into microservices.

17. Final TradeNest Definition

TradeNest is an AI-powered multi-vendor marketplace built as a NestJS modular monolith with a Next.js frontend, PostgreSQL/Prisma persistence, Zod contracts, secure JWT/bcrypt/RBAC authentication, an escrow-oriented payment state machine, and a separate Python/FastAPI intelligence service. The first implementation priority is a reliable end-to-end commerce and escrow workflow; recommendations and seller analytics follow, with advanced fraud and pricing models added after sufficient data and system maturity.

This blueprint is intended to be the working technical reference used when creating the repository, Prisma schema, NestJS modules, Zod contracts, API controllers, frontend screens, and test plan.